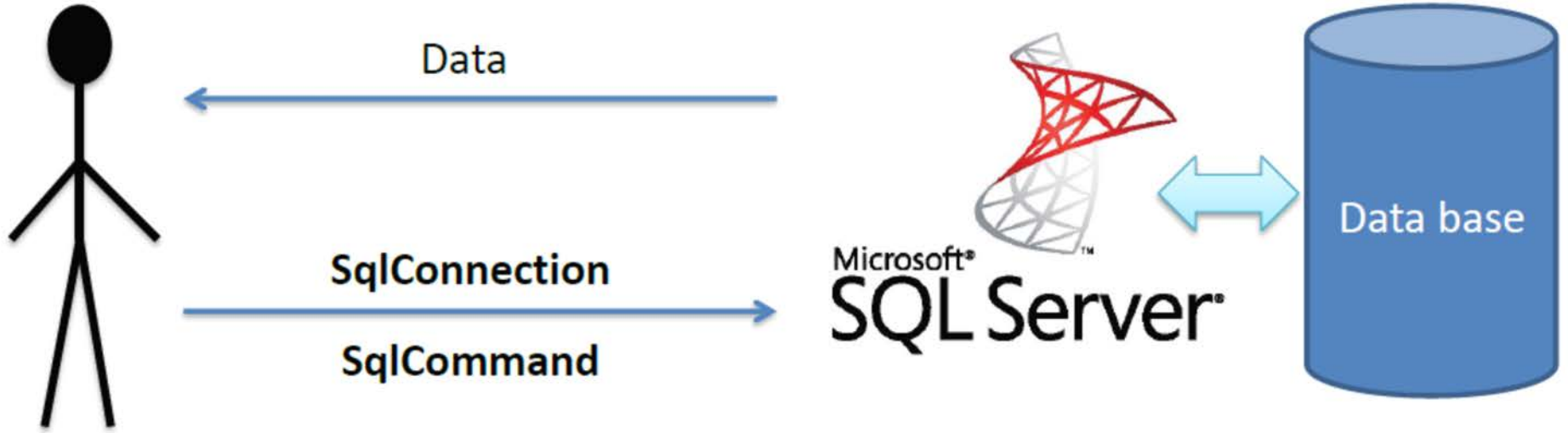
Создание и выполнение команд в ADO.NET

SqlCommand – объект подключаемой части технологии ADO.NET, позволяющий выполнять инструкции T-SQL над источником данных.

Для правильной работы объекту **SqlCommand** нужно предоставить подключение к источнику данных(экземпляр класса **SqlConnection**)



SqlCommand

Способы создания объекта **SqlCommand**:

1) Использование конструктора по умолчанию

```
SqlCommand cmd = new SqlCommand();
cmd.Connection = connection;
cmd.CommandText = "Some T-SQL Command";
```

2) Использование метода **CreateCommand()** объекта **SqlConnection**

```
cmd = connection.CreateCommand();
cmd.CommandText = "Some T-SQL Command";
```

3) Использование перегрузки конструктора с двумя параметрами

```
cmd = new SqlCommand("Some T-SQL Command", connection);
```



Независимо от способа создания объекта **SqlCommand**,
он должен получить экземпляр **SqlConnection**

SqlCommand

После создания объекта **SqlCommand** можно выполнять T-SQL инструкции над источником данных, для чего объект **SqlCommand** имеет ряд методов, позволяющих выполнять разные типы инструкций.

Методы объекта **SqlCommand** для выполнения команд:

1) **ExecuteNonQuery()** – этот метод предназначен для выполнения команд, не возвращающих значения. Такие команды могут выполнять следующие инструкции T-SQL: INSERT, DELETE, ALTER, DROP, CREATE.

2) **ExecuteScalar() –** этот метод предназначен для выполнения команд, возвращающих скалярные значения.

3) **EcecuteReader() –** этот метод предназначен для выполнения команд, возвращающих данные в табличном представлении(SELECT).

Объект SqlDataReader

Объект **DataReader** представляет собой аналог пожарного курсора только для чтения, позволяющий просматривать табличные данные, которые возвращает команда.

Получить объект **DataReader** можно с помощью метода **ExecuteReader()** объекта **Command**

```
SqlDataReader reader = cmd.ExecuteReader();
```



С помощью объекта **DataReader** можно просматривать только одну строку в один момент времени и только в одном направлении.

Описание объекта SqlDataReader

SqlDataReader имеет ряд методов, свойств и индексаторов для получения информации, пришедшей от источника данных
Ниже приведены некоторые из них.

Read() – этот метод возвращает значение true или false в зависимости от того, достигнут ли конец набора строк и при каждом его вызове перемещается к следующей строке.

FieldCount – свойство, позволяющее получить количество полей у строк, пришедших от источника.

GetName(int index) – метод, позволяющий получить имя поля по индексу

Для получения значений отдельных полей строк SqldataReader имеет две перегрузки индексатора с целочисленным индексом и строковым, а так же прототипизированные методы **Get<FieldType>(int index)** и **GetFieldValue<T>(int index)**

Асинхронное выполнение команд

При создании UI приложений, использующих объект **SqlCommand** выполнение команд следует производить асинхронно во избежание блокировки пользовательского интерфейса.

Специально для этого объект Command имеет методы асинхронного выполнения команд. Все методы для асинхронного выполнения команд имеют префикс **Async**.

```
await cmd.ExecuteNonQueryAsync();

await cmd.ExecuteReaderAsync();

await cmd.ExecuteScalarAsync();
```

Выполнение пакетных запросов

При создании **SqlCommand** свойству **CommandText** можно присвоить сразу несколько операторов T-SQL.

```
SqlCommand cmd = new SqlCommand();
cmd.Connection = connection;
cmd.CommandText = "SELECT * FROM Customers ; SELECT * FROM Employees;";
```

При использовании объекта **DataReader** для такой команды после просмотра данных первого набора строк нужно перейти к следующему набору с помощью метода **NextResult()** объекта **DataReader**.

Транзакцией называется выполнение последовательности команд (SQL- конструкций) в базе данных, которая либо фиксируется при успешной реализации каждой команды, либо отменяется при неудачном выполнении хотя бы одной команды.

```
BEGIN TRANSACTION;

DECLARE @Id int;
INSERT MyUserName VALUES ('TestName0','TestLName0');
SET @Id = @@IDENTITY;
INSERT MyUserTell VALUES (@Id,'(097)2224455');

COMMIT TRANSACTION;
```

Откат транзакции — это действие, обеспечивающее аннулирование всех изменений данных, которые были сделаны в теле текущей незавершенной транзакции.

```
BEGIN TRANSACTION;

DECLARE @Id int;
INSERT MyUserName VALUES ('TestName0','TestLName0');
SET @Id = @@IDENTITY;
INSERT MyUserTell VALUES (@Id,'(097)2224455');

ROLLBACK TRANSACTION;
```

TRANSACTION

При выполнении транзакций несколькими пользователями одной базы данных могут возникать следующие проблемы:

- **Dirty reads**
- **Non-repeatable reads**
- **Phantom reads**

Dirty reads – «грязное» чтение. Первый пользователь начинает транзакцию, изменяющую данные. В это время другой пользователь (или создаваемая им транзакция) извлекает частично измененные данные, которые не являются корректными.

Транзакция 1	Транзакция 2
SELECT f2 FROM tbl1 WHERE f1=1;	
UPDATE tbl1 SET f2=f2+1 WHERE f1=1;	
	SELECT f2 FROM tbl1 WHERE f1=1;
ROLLBACK WORK;	

Non-repeatable reads – неповторяемое чтение. Первый пользователь начинает транзакцию, изменяющую данные. В это время другой пользователь начинает и завершает другую транзакцию. Первый пользователь при повторном чтении данных (например, если в его транзакцию входит несколько инструкций SELECT) получает другой набор записей.

Транзакция 1	Транзакция 2
SELECT f2 FROM tbl1 WHERE f1=1;	SELECT f2 FROM tbl1 WHERE f1=1;
UPDATE tbl1 SET f2=f2+1 WHERE f1=1;	
COMMIT;	
	SELECT f2 FROM tbl1 WHERE f1=1;

Phantom reads – чтение фантомов. Первый пользователь начинает транзакцию, выбирающую данные из таблицы. В это время другой пользователь начинает и завершает транзакцию, вставляющую или удаляющую записи. Первый пользователь получит другой набор данных, содержащий фантомы – удаленные или измененные строки.

Транзакция 1	Транзакция 2
	SELECT SUM(f2) FROM tbl1;
INSERT INTO tbl1 (f1,f2) VALUES (15,20);	
COMMIT;	
	SELECT SUM(f2) FROM tbl1;



От неповторяющегося чтения оно отличается тем, что результат повторного обращения к данным изменился не из-за изменения/удаления самих этих данных, а из-за появления новых (фантомных) данных.

Для решения этих проблем разработаны четыре уровня изоляции транзакции в SQL:

- **Read uncommitted.**
- **Read committed.**
- **Repeatable read.**
- **Serializable.**

Read uncommitted. Транзакция может считывать данные, с которыми работают другие транзакции. Применение этого уровня изоляции может привести ко всем перечисленным проблемам.

Read committed. Транзакция не может считывать данные, с которыми работают другие транзакции. Применение этого уровня изоляции исключает проблему «грязного» чтения.

Repeatable read. Транзакция не может считывать данные, с которыми работают другие транзакции. Другие транзакции также не могут считывать данные, с которыми работает эта транзакция. Применение этого уровня изоляции исключает все проблемы, кроме чтения фантомов.

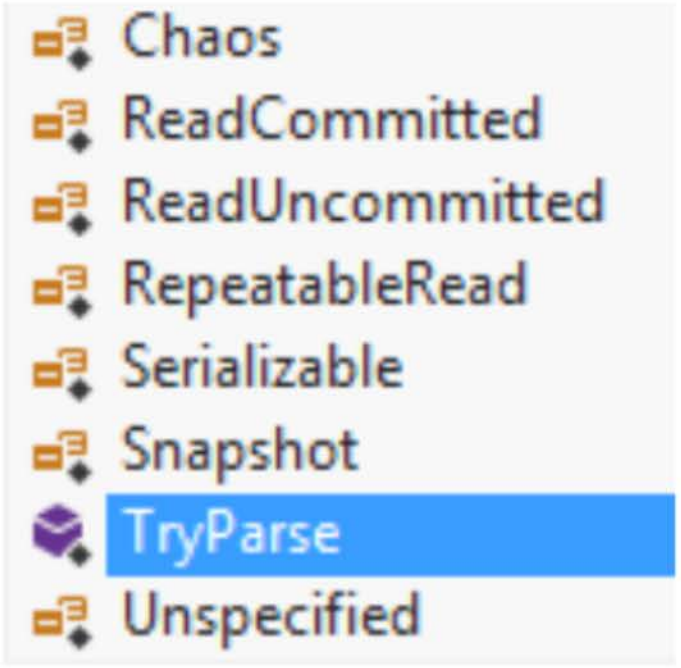
Serializable. Транзакция полностью изолирована от других транзакций. Применение этого уровня изоляции полностью исключает все проблемы.

Уровни изолированности транзакций, отличия реализации

Уровень изолированности	Dirty reads («грязное» чтение)	Non-repeatable reads (неповторяемое чтение)	Phantom reads (чтение фантомов)
READ UNCOMMITTED	ВОЗМОЖНО	ВОЗМОЖНО	ВОЗМОЖНО
READ COMMITTED		ВОЗМОЖНО	ВОЗМОЖНО
REPEATABLE READ			ВОЗМОЖНО
SERIALIZABLE			

В ADO.NET дополнительно поддерживаются еще два уровня:

- **Read uncommitted**
- **Read committed**
- **Repeatable read**
- **Serializable**
- **Chaos**
- **Unspecified**



Уровни изолированности транзакций. Применение в ADO.NET

```
SqlConnection connection = new SqlConnection(
ConfigurationManager.ConnectionStrings["SqlConectionString"].ConnectionString);
connection.Open();
SqlCommand myCommand = connection.CreateCommand();
// Создаем транзакцию
myCommand.Transaction = connection.BeginTransaction(IsolationLevel.Serializable);
// CommandText - текст запроса.
myCommand.CommandText = "DELETE FROM dbo.Products WHERE Description='ER203 Куртка'";
 Command.ExecuteNonQuery();
```

CommandText – текст запроса.

Метод **ExecuteNonQuery** применяется для выполнения запросов, не возвращающих данные, таких как UPDATE, INSERT и DELETE – они вносят изменения в таблицу базы данных, не возвращая ничего назад в результате выполнения.